

COMP7710

Programming Fundamentals

Today's lecture
Object-Oriented Programming

Course Convenor
Bernardo Pereira Nunes



Australian
National
University

All lectures are recorded and made available on Canvas

Agenda

- > Programming Paradigms
- > Classes and Objects
- > OO Concepts through the Employee example
- > Access Modifiers
- > Class Design Hints
- > Exercises



Programming Paradigms

Problem

> Given a list of integers, return the sum of the squares of the even numbers.

> **Example:** $[1, 2, 3, 4, 5] \rightarrow 2^2 + 4^2 = 20$

Solution 1: Imperative

```
% int sumEvenSquares(int[] xs) {  
    int sum = 0;  
    for (int x : xs) {  
        if (x % 2 == 0) {  
            sum += x * x;  
        }  
    }  
    return sum;  
}
```

Imperative programming **focuses on HOW a computation is performed**. The program is a **sequence of commands** that explicitly manipulate state (variables) step by step.

manual state management (e.g., sum variable updates)

step-by-step instructions telling the computer exactly how to achieve the result



Programming Paradigms

Problem

> Given a list of integers, return the sum of the squares of the even numbers.

> **Example:** $[1, 2, 3, 4, 5] \rightarrow 2^2 + 4^2 = 20$

Solution 2: Procedural

```
% boolean isEven(int x) { return x % 2 == 0; }  
% int square(int x) { return x * x; }
```

```
% int sumEvenSquares(int[] xs) {  
    int sum = 0;  
    for (int x : xs) {  
        if (isEven(x)) sum += square(x);  
    }  
    return sum;  
}
```

Procedural programming builds on the imperative style by **organising code into procedures (functions/methods)**. The goal is **better structure, reuse, and readability**, while still following step-by-step execution.

procedures perform a small, well-defined task

loop still controls execution and state changes



Programming Paradigms

Problem

> Given a list of integers, return the sum of the squares of the even numbers.

> **Example:** $[1, 2, 3, 4, 5] \rightarrow 2^2 + 4^2 = 20$

Solution 3: Functional

```
% int sumEvenSquares(int[] xs) {  
    return java.util.Arrays.stream(xs)  
        .filter(x -> x % 2 == 0) // WHAT: filter evens  
        .map(x -> x * x) // WHAT: square them  
        .sum(); // WHAT: sum results  
}
```

immutable data transformations

Functional programming focuses on **WHAT transformation is applied to data**, rather than how to mutate state. Computation is **expressed as compositions of functions**, often avoiding explicit loops and mutable variables. It emphasises expressions over instructions.

Function composition (filter → map → reduce)



Programming Paradigms

Problem

> Given a list of integers, return the sum of the squares of the even numbers.

> **Example:** $[1, 2, 3, 4, 5] \rightarrow 2^2 + 4^2 = 20$

Solution 4: Declarative (e.g. SQL)

```
// assuming values are in a table nums(n)
% SELECT SUM(n*n) AS sum_even_squares
  FROM nums
 WHERE n % 2 = 0;
```

Declarative programming describes **WHAT** result is desired, **not the steps to obtain it**. The execution strategy is left to the underlying system or engine.

programmer expresses **constraints and relationships**, and the system decides *how* to compute the result



Programming Paradigms

Object-oriented (OO) programming **structures** programs around **objects**, which bundle together **data (state)** and **behaviour (methods)**. The focus is on **modelling responsibilities and interactions**.

Problem

> Given a list of integers, return the sum of the squares of the even numbers.

> **Example:** $[1, 2, 3, 4, 5] \rightarrow 2^2 + 4^2 = 20$

Solution 5: Object-Oriented

```
% class NumberList { // Pure data container
    private final List<Integer> numbers;
    public NumberList(List<Integer> numbers) {
        this.numbers = new ArrayList<>(numbers);
    }
    ...}
```

```
class NumberOperations { // Behaviour class (Pure Logic) }
    public static boolean isEven(int number) {
        return number % 2 == 0;
    }
    public static int square(int number) {
        return number * number;
    }
}
```

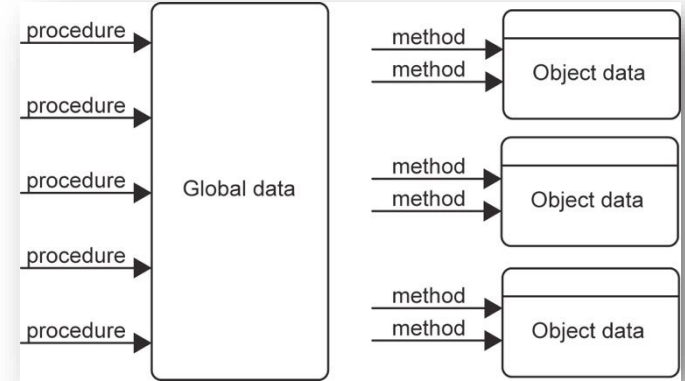
```
class SumCalculator { // Behaviour (Pure Logic)
    public static int sumOfSquaresOfEvens(NumberList numberList) {
        int sum = 0;
        for (int number : numberList.getNumbers()) {
            if (NumberOperations.isEven(number)) {
                sum += NumberOperations.square(number);
            }
        }
        return sum;
    }
}

public class Main { // Usage
    public static void main(String[] args) {
        NumberList data = new NumberList(Arrays.asList(1, 2, 3, 4, 5));
        int result = SumCalculator.sumOfSquaresOfEvens(data);
        System.out.println(result); // 20
    }
}
```



Programming Paradigms

- > For **small problems**, **procedural decomposition** works well
- > For **larger systems**, **object-oriented design** scales better
- > A **complex system** (e.g., a web browser) might **require thousands of procedures** operating on shared global data, which is **hard to understand and debug**
- > In an **object-oriented design**, the same system can be **structured as many small classes**, each with a limited number of methods
- > OO design improves readability, limits the scope of data access, and makes bugs easier to locate



Procedural vs. OO programming

2,000 procedures

vs

100 classes (with 20 methods)



Classes and Objects

- > A **class** is a **blueprint** (or template) that defines:
- >> **Fields** → what data an object stores
- >> **Methods** → what an object can do

A class does **not** represent a real thing by itself: it describes one!

Template

(specifies how objects are made)



Instance

(objects are the cookies themselves)



Image source: Natalia Campbell (Youtube)

Cookie Cutters



Image source: starpackproducts.com



Classes and Objects

> In more details now... let's start with objects this time!

> *"An object is a software bundle of related state and behaviour"*

objects have

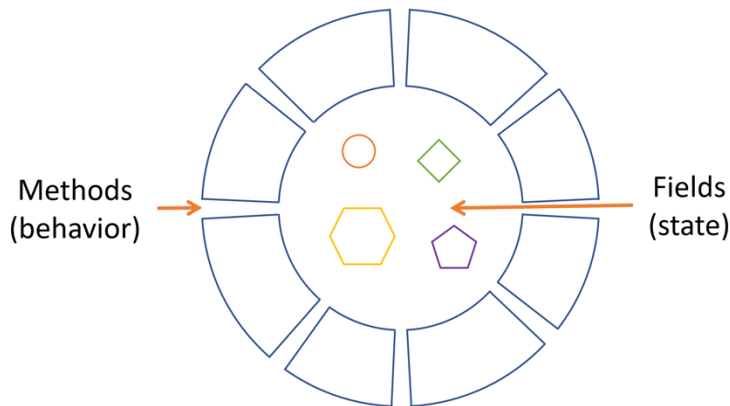
> Thinking in objects starts with identifying state and behaviour in real-world entities

> For example, a bicycle has:



>> **State:** current gear, pedal cadence, speed

>> **Behaviour:** change gear, adjust cadence, apply brakes



Source: <https://dev.java/learn/oop/>



Classes and Objects

< Exercise >

> Look around you and pick any real-world object

>> For each object that you see, ask yourself the following two questions:

What possible states can this object be in?

What possible behaviour can this object perform?



Reflection questions

How complex is the object?

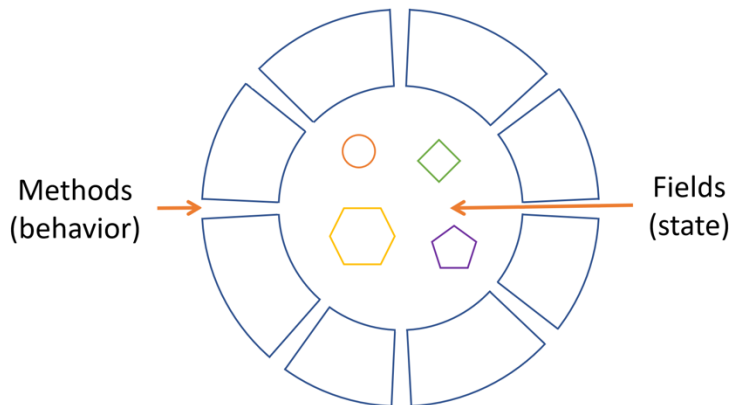
At what level of detail did you model it?

Did you identify objects that contain other objects?

change gear, adjust
cadence, apply brakes



current gear, pedal
cadence, speed



Source: <https://dev.java/learn/oop/>



Classes and Objects

> An **object** stores its **state** in **fields** and **exposes its behaviour** through **methods**

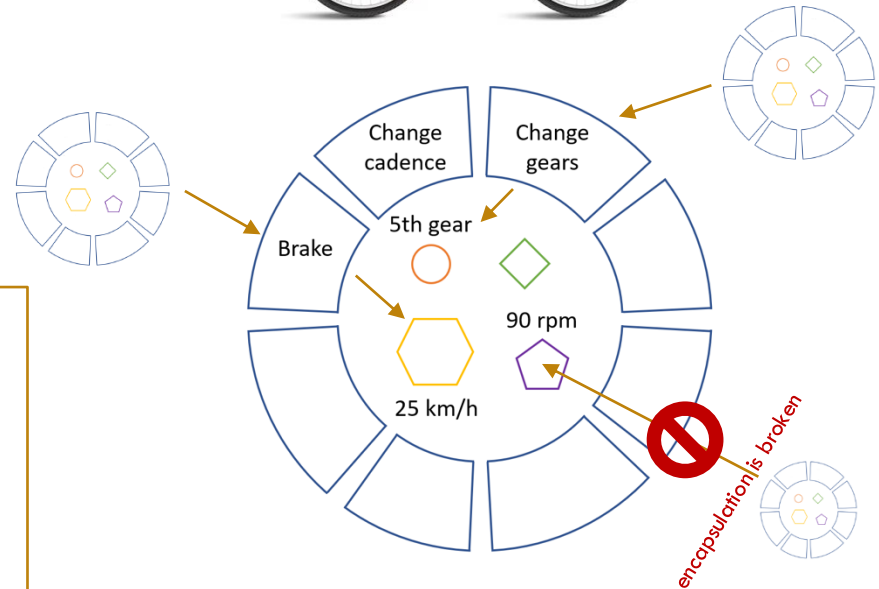
> **Methods** operate on an object's **internal state** and serve as the primary mechanism **for object-to-object communication**

Hiding internal state and requiring all interaction to be performed through an object's methods is known as data **encapsulation** (a fundamental principle of object-oriented programming)

Black box behaviour: Methods should never directly access instance fields in a class other than their own. Programs should interact with object data through the object's methods

change gear, adjust
cadence, apply brakes

current gear, pedal
cadence, speed



Source: <https://dev.java/learn/oop/>



Classes and Objects

> By **encapsulating state** (e.g., speed, pedal cadence, gear) and **exposing controlled methods to modify it**, an object governs how it can be used by the outside world

>> For example, **if a bicycle has six gears**, a gear-change **method can enforce valid values (1–6)**

Benefits of bundling code into objects include:

Modularity: Objects can **be developed and maintained independently and reused** across the system

Information hiding (encapsulation): **Interaction occurs through methods**, concealing internal implementation details

Code reuse: Existing, well-tested objects can be incorporated, **reducing development and debugging effort**

Pluggability and ease of debugging: **Faulty objects can be replaced without affecting the rest of the system**, analogous to replacing a component rather than an entire machine

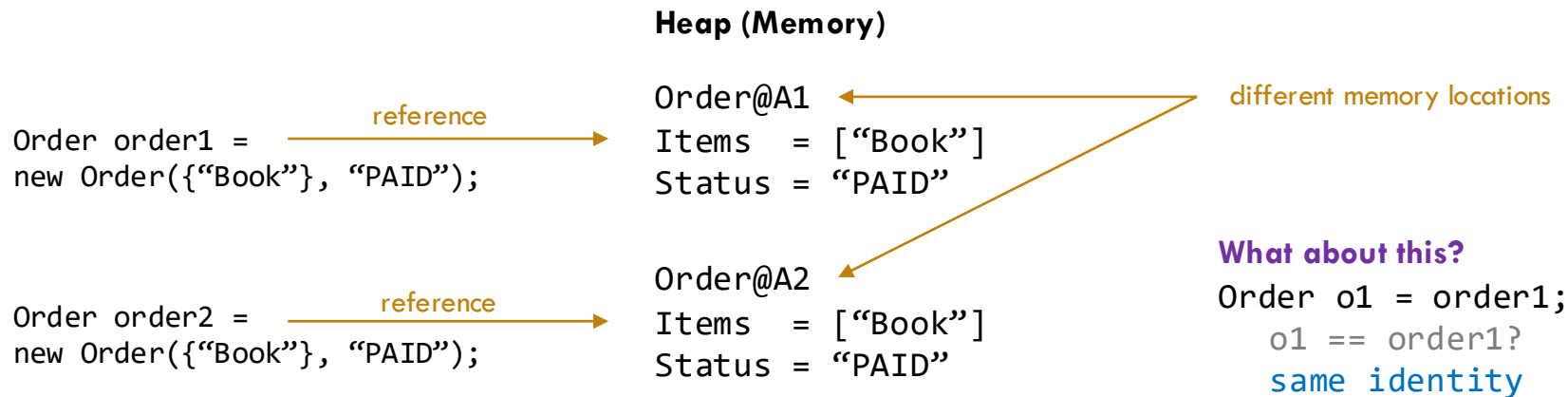
Source: <https://dev.java/learn/oop/>



Classes and Objects

> Object Identity vs Object State

>> Even if two objects have identical state, they are still distinct objects because each has its own identity



Identity is tied to object identity in memory, not its data!

`order1 == order2` (false, different identities)

`order1.equals(order2)` (possibly true depending on implementation/content)



Classes and Objects

Identifying Classes

- > Procedural programs start the design process at the top (with the main function)
- > When designing an object-oriented system, there is no “top”!

Where do we begin then?

look for	in problem analysis	corresponds to
Nouns	Objects, people, concepts, things	Classes
Verbs	Actions, operations, behaviours	Methods

Practical Approach

Read the problem statement

Circle all nouns → Potential classes

Underline all verbs → Potential methods

Filter and refine based on system requirements

Example

Customer, Order, Product → Classes (nouns)

calcTotal(), processPayment(), updateInventory() → Methods (verbs)



Classes and Objects

Practical Approach

Read the problem statement

Circle all nouns → Potential classes

Underline all verbs → Potential methods

Filter and refine based on system requirements

Identifying Classes

look for	in problem analysis	corresponds to
Nouns	Objects, people, concepts, things	Classes
Verbs	Actions, operations, behaviors	Methods

< Exercise >

A library manages books. Members borrow and return books. Each book has title, author, and ISBN. Librarians can add new books and remove damaged ones. Members can search for books by title.



Classes and Objects

< Exercise >

A library manages books. Members borrow and return books. Each book has title, author, and ISBN. Librarians can add new books and remove damaged ones. Members can search for books by title.

Nouns

Potential Classes:

Library: The main system/organisation

Book: The primary resource

Member: Library users

Librarian: Staff/administrator

Attributes/Properties:

Title: Attribute of Book

Author: Attribute of Book

ISBN: Attribute of Book

Books: Plural of Book, might be collection in Library

Members: Plural of Member, collection in Library

Additional Considerations:

Search - Could be a class or method

Damage - State/condition attribute



Classes and Objects

< Exercise >

A library manages books. Members borrow and return books. Each book has title, author, and ISBN. Librarians can add new books and remove damaged ones. Members can search for books by title.

Verbs

Potential Methods:

Member Class:

borrow → borrowBook(Book book)
return → returnBook(Book book)
search → searchByTitle(String title) - Catalog/Library class
...

Librarian Class:

manages → manageCollection()
add → addBook(Book book)
remove → removeBook(Book book)
...

Book Class

has (title, author, ISBN) → getTitle(), getAuthor(), getISBN()
is available → isAvailable()
is damaged → isDamaged()

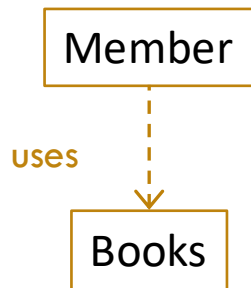


Classes and Objects

Relationships between Classes (simplified)

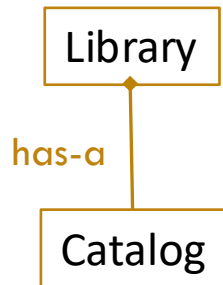
> The **most common** relationships between classes are:

Dependency



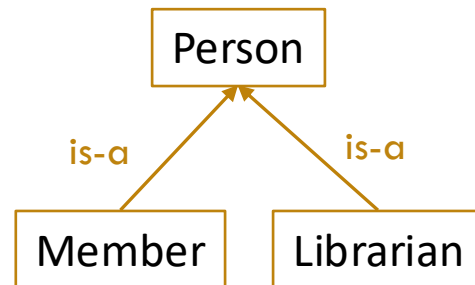
```
class Member {  
    public void borrowBook(Book book) {  
        if (book.isAvailable()) {...}  
    }  
    ...  
}
```

Aggregation



```
class Library {  
    private Catalog catalog;  
    ...  
}
```

Inheritance



```
abstract class Person {...}  
class Member extends Person {...}  
class Librarian extends Person {...}
```



Creating a Class in Java using the Bicycle Example

```
public class Bicycle {
```

```
    int cadence = 0;
```

```
    int speed = 0;
```

```
    int gear = 1;
```

```
    public void changeCadence(int newCadence) {  
        cadence = newCadence;
```

```
    }
```

```
    public void changeGear(int newGear) {  
        gear = newGear;
```

```
    }
```

```
    public void speedUp(int increment) {  
        speed += increment;
```

```
    }
```

```
    public void applyBreaks(int decrement) {  
        speed -= decrement;
```

```
    }
```

```
}
```

—————→ The file name must match the class name (e.g., "Bicycle.java")

```
    public void printStates() {  
        IO.println("cadence:" +  
            cadence + " speed:" +  
            speed + " gear:" + gear);  
    }
```



Creating a Class in Java using the Bicycle Example

```
class BicycleDemo {  
    public static void main(String[] args) {  
        Bicycle bike1 = new Bicycle();  
        Bicycle bike2 = new Bicycle();  
  
        bike1.changeCadence(50);  
        bike1.speedUp(10);  
        bike1.changeGear(2);  
        bike1.printStates();  
  
        bike2.changeCadence(50);  
        bike2.speedUp(10);  
        bike2.changeGear(2);  
        bike2.changeCadence(40);  
        bike2.speedUp(10);  
        bike2.changeGear(3);  
        bike2.printStates();  
    }  
}
```

main is the entry point of a Java program

Create two different Bicycle objects

Invoke methods on bike 1

Invoke methods on bike 2



Classes and Objects

> Using predefined Classes

>> In Java, **everything is built around classes**

>> We have already used several classes, but not all illustrate full object-oriented features

> Example: Math class

>> Methods such as `Math.random()` can be used without knowing their implementation

>> This demonstrates **encapsulation: only method names and parameter types matter**

>> The Math class **encapsulates functionality only**

>> It has **no instance data**, so:

>>> No objects need to be created

>>> No instance fields need to be initialised

`Math.random()` is a **static** method in the `java.lang.Math` class that returns a pseudorandom double value greater than or equal to 0.0 and less than 1.0.



Classes and Objects

> Working with Objects in Java

- >> Objects must be constructed before they can be used
- >> Construction defines the object's initial state
- >> Once created, you invoke methods to operate on the object

> Constructors

- >> In Java, objects are created using constructors
- >> A constructor is a special method used to create and initialise an object
- >> A constructor always has the same name as the class

> Example: Date class

- >> The constructor is called Date

```
% new Date();  
% IO.println(new Date()); //test using jshell
```

the **new** operator allocates memory for the object and calls the constructor; in the case of the Date class, the resulting Date object is initialised to the current date and time

- >> A Date object represents a specific point in time (e.g., December 31, 1999, 23:59:59 GMT)
- >> The standard Java library provides the Date class

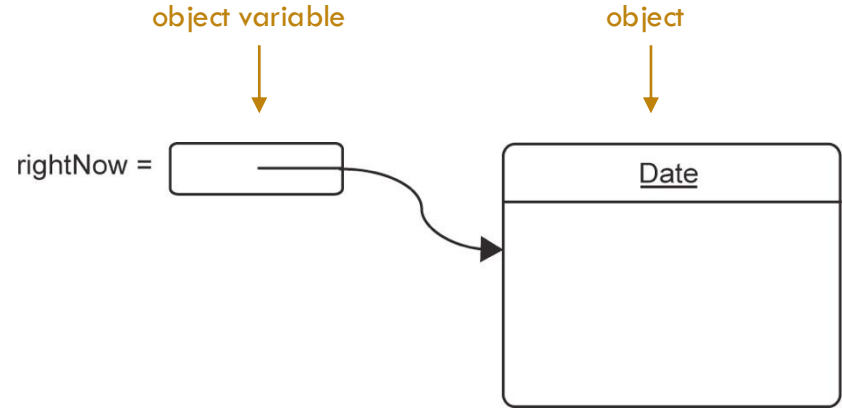


Classes and Objects

> Object and Object Variable

```
% Date rightNow;
% rightNow = new Date();
```

object variable
is declared (null)



Creating an Object variable `rightNow` that refers to the newly constructed object



Classes and Objects

> The Date Class: A Point in Time

- >> An **instance of Date** represents a specific point in time
- >> Internally, **time is stored as milliseconds since the epoch**
- >>> *Epoch = 00:00:00 UTC, January 1, 1970*
- >> This **representation is machine-oriented** (hidden from the user/encapsulated)

> Why Date is limited for humans

- >> **Humans** reason about dates **using calendars** (e.g., December 31, 1999)
- >> Calendars **vary across cultures** (Gregorian, Chinese, Hebrew, etc.).
- >> **A raw point in time is not sufficient for calendar-based reasoning**

> Separating Concepts: Good OO Design

- >> Java separates: **Time measurement** → **Date** and **Calendar dates** → **LocalDate**
- >> This clarifies intent, improves API usability, and reflects good OO design



Classes and Objects

> The LocalDate Class

- >> **LocalDate** represents a calendar date: *year, month, day*
- >> Uses the Gregorian Calendar
- >> This **representation is human-oriented**

> Creating LocalDate Objects

- >> **LocalDate objects are not created using constructors**
- >> **Java uses static factory methods:**

```
% LocalDate.now(), // vs new Date()  
% LocalDate.of(1999, 12, 31);  
% LocalDate newYearsEve = LocalDate.of(1999, 12, 31);  
% newYearsEve.getYear(); //provides methods to access its state  
% LocalDate aThousandDaysLater = newYearsEve.plusDays(1000);
```

Factory methods

- > Improve readability (e.g., name conveys intent)
- > Hide implementation details
- > Control object creation
- >> validate inputs, reject invalid states
- >> decide what to return
- >>> new object, cached object, a subtype

Objects provide behaviour, not just data access; Computes a new date; Returns a new object.

> What happens to the variable newYearsEve after calling plusDays?

- > It **creates and returns a new LocalDate object**; new object is **assigned to aThousandDaysLater**; the **original object remains unchanged** (plusDays is a non-mutating method (immutability))



Classes and Objects

Methods that only access objects without modifying them are sometimes called accessor methods

> JShell

```
% LocalDate myBirthday = LocalDate.of(2026, 3, 25)
% LocalDate aHundredDaysLater = myBirthday.plusDays(100);
```

The `plusDays` method yields a new `LocalDate` object, which is then assigned to the `aHundredDaysLater` variable. The **original object remains unchanged**.
The **`plusDays` method does not mutate the object**.

```
% GregorianCalendar myBirthday = new GregorianCalendar(2026, 2, 25);
% GregorianCalendar aHundredDaysLater = myBirthday.add(Calendar.DAY_OF_MONTH, 100);
% myBirthday.get(Calendar.YEAR);
% myBirthday.get(Calendar.MONTH); // (Months: 0-11)
% myBirthday.get(Calendar.DAY_OF_MONTH);
```

`GregorianCalendar.add` method is a mutator method; `.add` returns nothing (void)



Classes and Objects

> Key Difference between Constructors and Factory methods

> Constructors

- >> Always create a new object
- >> Must return the class type

> Factory methods

- >> May return an existing object
- >> May return a different implementation
- >> Provide more flexibility in design
- >> Factory methods give the class control over its own instances



<Exercises>

Which paradigm focuses primarily on HOW a computation is performed step by step?

- A. Functional
- B. Declarative
- C. Imperative
- D. Object-Oriented



<Exercises>

Why does Object-Oriented design scale better for large systems?

- A. It avoids methods entirely
- B. It replaces loops with streams
- C. It groups data and behaviour into small, focused classes
- D. It removes the need for testing



<Exercises>

Which statement best describes the relationship between a class and an object?

- A. A class is a specific instance of an object
- B. A class describes how objects are created
- C. Objects contain classes
- D. Classes exist only at runtime



<Exercises>

Which of the following is state, not behaviour?

- A. `changeGear()`
- B. `speedUp()`
- C. `currentSpeed`
- D. `applyBrakes()`



<Exercises>

What is encapsulation in OOP?

- A. Storing state in public fields
- B. Hiding internal state and exposing behaviour through methods
- C. Allowing direct access to object fields from other classes
- D. Using only static methods



<Exercises>

If two objects have identical state, are they the same object?

- A. Yes, because their data is the same
- B. No, unless they are primitives
- C. Yes, if they are in the same memory location
- D. No, because each object has its own identity



<Exercises>

What does `LocalDate.of(1999, 12, 31)` return?

- A. A mutable Date object
- B. A new immutable LocalDate object
- C. A static field
- D. A reference to an existing shared object



< Exercise >

> Write a Java program that prints a calendar for the current month, marking today with an asterisk (*)

> Key requirements

>> Compute:

>>> Length of the month (28-31 days)

>>> Weekday of the 1st day of the month

>>> Weekday for each date to align the layout

>>> Mark the current day with *

> Steps

1. Read and trace the reference solution (Book **Ch 4.2.3** + course repo)

2. Go **line by line** and mentally explain what each line does

3. Close the solution and **re-implement from memory**

>> Use the Java docs when needed (looking for methods, definitions)

4. Test your program on different dates/months

no comments



with lots of
comments



You are expected to consult the official Java documentation to discover which methods help solve the task

Mon	Tue	Wed	Thu	Fri	Sat	Sun
						1
2	3	4	5	6	7	8
9	10	11	12	13	14	15
16	17	18	19	20	21	22
23	24	25	26*	27	28	29
30						

Hints (what to look up in the docs)

> `LocalDate.now()`

> Getting the first day of the month

> Month length (number of days)

> Day-of-week for a date

> Advancing dates day-by-day



Dissecting a small program

The simplest form for a class definition in Java is:

```
class ClassName {  
    field1  
    field2  
    . . .  
    constructor1  
    constructor2  
    . . .  
    method1  
    method2  
    . . .  
}
```

```
class Employee {  
    // instance fields  
    private String name;  
    private double salary;  
    private LocalDate hireDay;  
  
    // constructor  
    Employee(String n, double s, int year, int month, int day) {  
        name = n;  
        salary = s;  
        hireDay = LocalDate  
    }  
  
    // a method  
    String getName() {  
        return name;  
    }  
  
    // more methods  
    . . .  
}
```

state (instance fields)
state changes only through
well-defined methods

constructor
method for object initialisation

Behaviour (methods)
Access state
Modify state via controlled logic

workhorse class



The Employee Class

> The Employee class models a **business entity** with clear responsibilities

> **Employee is a business entity**

>> it represents a **real concept in the problem domain** (a company employee)

>> not a technical helper, not a utility, not a temporary data structure

Abstraction
we ignore irrelevant details

> **Models**

>> the class captures state (e.g., name, salary, role)

>> and behavior (e.g., getSalary(), applyRaise(), changeRole())

> **Clear responsibilities**

>> the class is responsible only for things that belong to an employee

>> salary logic lives in Employee

>> personal details live in Employee

>> it does not handle input/output, databases, or UI

Single Responsibility Principle

Employee class knows what an employee is and how an employee changes



The Employee Class

> Constructor

>> a constructor is a special method that is **automatically executed when an object is created**

>> its purpose is to **initialize the instance fields** of a new object

> an instance of the Employee class can be created like this:

```
% new Employee("James Bond", 100000, 1950, 1, 1);
```

← same name as the class

> constructors cannot be used to reset an object's fields:

```
% james.Employee("James Bond", 250000, 1950, 1, 1);
```

> a constructor has the **same name as the class**

> a class can have more than one constructor (more later)

> a constructor can have **zero, one, or more parameters**

> a constructor **has no return value**

> a constructor can **only be called with new**

Basic declaration of a constructor

```
Class Employee { ...  
    Employee(String n, double s, int  
    year, int month, int day) {  
        ...  
    }  
}
```



The Employee Class

Constructors: Be careful: local variables

```
Class Employee {  
    private final String name;  
    private final double salary;  
    Employee(String n, double s, int year, int month, int day) {  
        String name = n;  
        double salary = s;    } shadowing: local variables shadow fields  
    }  
}
```

→ java: variable name might not have been initialised

```
Class Employee {  
    private final String name;  
    private final double salary;  
    Employee(String n, double s, int year, int month, int day) {  
        name = n; // this.name  
        salary = s; // this.salary  
    }  
}
```



The Keyword 'this' (common cases)

> **'this'** refers to the current object: the instance whose method or constructor is being executed

```
public class Employee {  
    private double salary;  
  
    public Employee(double salary) {  
        this.salary = salary;  
    }  
}
```

instance field constructor parameter

Shadowing: parameter names are the same as field names

```
public double getSalary() {  
    return this.salary;  
}
```

'this' is optional, used for clarity (no conflicts and is accessing current object's field)

```
// Full constructor  
public Employee(String name, double salary, LocalDate hireDate) {  
    this.name = name;  
    this.salary = salary;  
    this.hireDate = hireDate;  
}  
  
// Convenience constructor - chains to full constructor  
public Employee(String name, double salary) {  
    this(name, salary, LocalDate.now()); // Calls the full constructor  
    // MUST be first statement in constructor  
}  
  
// Default constructor  
public Employee() {  
    this("Unknown", 0.0, LocalDate.now()); // Calls the 3-arg constructor  
}
```

Explicit constructor invocation: Calling another constructor in the same class; prevents code duplication in constructors; the invocation of another constructor must be the first line in the constructor (Java 25 allows validation/normalisation (see Ch 4.6.6 in Core Java Vol 1 for more details))



The Employee Class

> Declaring Local Variables with var

>> The var identifier in Java is used for local variable type inference, a feature introduced in Java 10.

>> It allows developers to declare local variables without explicitly specifying their data type, letting the **compiler infer the type from the value used to initialise the variable**

```
% var v; // ERROR
```

```
% var v = "Bernardo"; // Mandatory Initialiser: Type must be inferable from right-hand side
```

> “**var**” cannot be used for **fields**, **method parameters**, and **method return types**

> Although it seems simple, **var** involves many design decisions:

>> let's take a look at <https://openjdk.org/projects/amber/guides/lvti-faq>

> Quick summary

>> No runtime overhead: **purely compile-time feature**

>> Error messages show inferred type

>> Improves maintainability when types change / avoid using with numeric literals to prevent confusion

>> It may reduce code clarity (use when type is obvious to readers)



The Employee Class

> Working with null References

>> Object variable can hold a reference to an object or null (primitives cannot be null)

>> Null shows the absence of an object

>> and calling methods on null causes an exception (more later in the course)

>>> **NullPointerException**

```
% var name = null;
```

← ERROR: cannot infer type: variable initializer is 'null'

```
% String name = null;
```

← Object variable is declared

```
% String s = name.toString();
```

← Throws NullPointerException (the most common runtime exception)
ERROR: Cannot invoke "String.toString()" because "name" is null

> Handling nulls

```
% name = Objects.requireNonNullElse(n, "unknown");
```

← Business requirements: is "unknown" name acceptable?

```
% name = Objects.requireNonNull(n, "Name cannot be null");
```

← "Fail fast" principle: detect errors as close to source as possible

> **requireNonNullElse**: allows object creation and ensures non-null state

> **requireNonNull**: clear error message; error occurs at the source; easier debugging

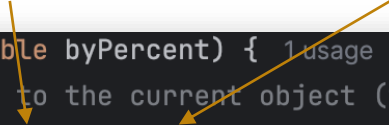


The Employee Class

> Implicit and Explicit Parameters

>> Methods operate on objects and access their instance fields

```
void raiseSalary(double byPercent) {  
    // 'this' refers to the current object (the one the method was called on)  
    this.salary += (this.salary * byPercent / 100);  
}
```



>> For example, the method `raiseSalary` sets a new value for the salary instance field in the object on which this method is invoked.

> Now, consider the call:

```
Employee empl = new Employee( n: "James Bond", s: 100000, year: 1950, month: 1, day: 1);  
empl.raiseSalary( byPercent: 5); //explicit argument: 5    // implicit argument: empl
```

>> the `raiseSalary` method is called with two arguments:

>>> the **implicit argument** (the **object** of type `Employee` that **appears before the method name**)

>>> the **explicit argument**, the **number inside the parentheses** after the method name



The Employee Class

> Accessor Methods and Encapsulation

> Examples of accessor methods

```
% String getName() { return name; }  
% double getSalary() { return salary; }  
% LocalDate getHireDay() { return hireDay; }
```

>> they simply return the values of instance fields (name, salary, hireDay).

>> they are sometimes called **field accessors**

Wouldn't it be easier to make the name, salary, and hireDay fields directly accessible, instead of having separate accessor methods?

> **accessor methods** preserve encapsulation, centralizes control (e.g., the salary field can only be changed by the raiseSalary method), limits debugging scope, prevents uncontrolled modification

> **direct field access** makes bugs harder to track and may break invariants (condition that must always be true for an object to be in a valid state, e.g., a salary can never be negative)



The Employee Class

> > Accessor Methods and Encapsulation

> Let's say there is a new internal representation for the field name:

>> the `String name` was refactored into `String firstName` and `String lastName`

> What happens to the original `getName` method?

```
% String getName() { return name; }
```

> the `getName` method can be changed to return

```
% String getName() { return firstName + " " + lastName; }
```

> and this change is completely invisible to the remainder of the program (do not break clients)



The Employee Class

at first glance, this seems harmless, but it can break encapsulation as the Date class provides mutator methods (e.g., setTime).

> Accessor Methods and Encapsulation

>> We must be careful not to expose or return mutable objects

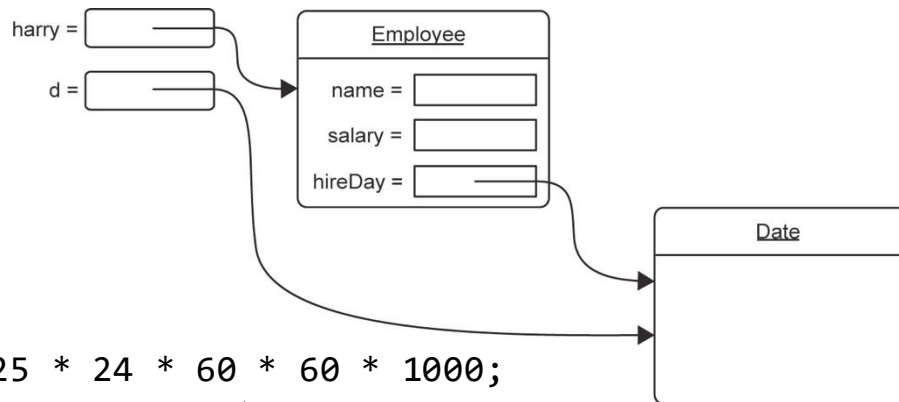
>> For example, if instead of using `LocalDate` we used `Date`, the accessor method might look like this:

```
% Date getHireDay() {  
    return hireDay;  
}
```

>> Let's see why the Date class breaks encapsulation:

```
% Employee harry = ...;  
% Date d = harry.getHireDay();  
% double tenYearsInMilliseconds = 10 * 365.25 * 24 * 60 * 60 * 1000;  
% d.setTime(d.getTime() - (long) tenYearsInMilliseconds); // +10 years
```

>> This allows external client code to modify the private state of the Employee object



The Employee Class

> Mutator Methods and Encapsulation

> A mutator method (also called a setter) can enforce class rules by performing:

>> validation and error checking

>> controlled updates to object state

>> In contrast, **direct field assignment** may **bypass** these checks and lead to invalid object states

>> For example, a setSalary method can ensure that the salary is never negative:

```
% void setSalary(double salary) {  
    if (salary < 0) {  
        throw new IllegalArgumentException("Salary must be non-negative");  
    }  
    this.salary = salary;  
}
```

Prevents invalid values (< 0), protect class invariants,
validation logic is centralized, easy to maintain



The Employee Class

> Private Methods and Encapsulation

> All instance fields should be private to protect the internal state of a class

```
% String name; // should be declared as private String name;
```

> **helper methods** *used only to support an internal implementation* **should also be private**

>> tightly coupled to the current data representation

>> unsuitable for the public interface

> In Java, a method is declared private by using the private access modifier:

```
% private void calculateInflation(double inflationIndex) { ... }
```

> Note that private methods are not part of the class contract. They can be changed/removed without affecting external code!

Only expose what clients need; keep implementation details private



The Employee Class

> Final instance fields

> The name field of the Employee class may be declared as `final` because it never changes after the object is constructed: there is no `setName` method

```
% class Employee {  
    private final String name;  
    . . .  
}
```

> For mutable classes, the `final` modifier can be confusing.

>> For example, consider a field

```
% private final StringBuilder evaluations; // StringBuilder is mutable
```

> that is initialized in the Employee constructor as

```
% evaluations = new StringBuilder();
```

> The `final` keyword merely means that the `object reference stored in the evaluations variable will never again refer to a different StringBuilder object`. However, the object can be mutated:

```
% evaluations.append(LocalDate.now() + ": Gold star!\n");
```



The Employee Class

> Static Fields and Methods

>> A **static field** (also called **class variable**) is a field declared with the **static modifier**. There is exactly **one copy of this field**, regardless of how many instances of the class are created

>> **Static fields** are loaded when class is first loaded into JVM

>> **Instance fields** are allocated when objects are created (in heap)

```
public class Employee {  
    // INSTANCE members (one per object)  
    private String name;  
    private double salary;  
  
    // STATIC members (one per class)  
    private static int count = 0;  
    public static final int MAX_VACATION_DAYS = 25;  
  
    public static int getEmployeeCount() {  
        return employeeCount; // Access static field  
    }  
}
```

% int total = Employee.getEmployeeCount();

Employee Class

Static Area – Class Level (shared)

count = 2
MAX_VACATION_DAYS = 25

Objects – Instance Level

Object 1: Empl@1234

name = "John"
salary = "50000"

Object 1: Empl@4321

name = "Mary"
salary = "70000"



The Employee Class

> Static Fields and Methods

>> A **static constant** is a static field declared as final

>> it represents a value that is shared across all instances and cannot be changed after initialization

> Common uses of constants

>> Replace hard-coded values (e.g., MAX_VACATION_DAYS instead of 25)

>> Configuration values (application-wide settings)

>> Mathematical constants (PI, E, etc.)

> Best practices

>> Constants should be immutable (primitives or immutable objects)

>> Use UPPER_CASE naming convention for clarity

```
public class Employee { ...  
    public static final int MAX_VACATION_DAYS = 25;  
    ...  
    for (int i = 0; i < 25; i++) {...}  
}
```

for (int i = 0; i < MAX_VACATION_DAYS; i++) {...}

Employee Class

Static Area – Class Level (shared)

count = 2

MAX_VACATION_DAYS = 25

Objects – Instance Level

Object 1: Empl@1234

name = "John"

salary = "50000"

Object 1: Empl@4321

name = "Mary"

salary = "70000"



The Employee Class

> Static Fields and Methods

- >> a **static method** is a method **declared with the static modifier**
- >> it **belongs to the class** rather than any object instance and **can be invoked without creating an object**
- >> it cannot use 'this' keyword
- >> it cannot directly access instance fields or methods
- >> it can only call other static methods and access static fields

```
public class MathUtils {  
    // Static method: no 'this' reference  
    public static double calculateBonus(double salary, double percentage) {  
        // No instance fields or methods can be accessed directly  
        return salary * percentage / 100;  
    }  
  
    // Instance method: requires object  
    public double getTaxAmount() {  
        return this.salary * TAX_RATE; // 'this' refers to object  
    }  
}
```



The Employee Class

> Static Fields and Methods

>> Use Static Methods for:

```
public class Employee { ...  
    public static Employee createDefaultEmployee(String name) {  
        return new Employee(name, 50000, LocalDate.now());  
    }  
  
    public static boolean isValidSalary(double salary) {  
        return salary >= 0 && salary <= 1_000_000;  
    }  
  
    public static double calculateNetSalary(double gross, double taxRate) {  
        return gross * (1 - taxRate);  
    }  
  
    public static int getTotalEmployees() {  
        return Employee.employeeCount;  
    }  
...}
```

Factory methods

Utility/helper methods

Mathematical operations

Access to static fields

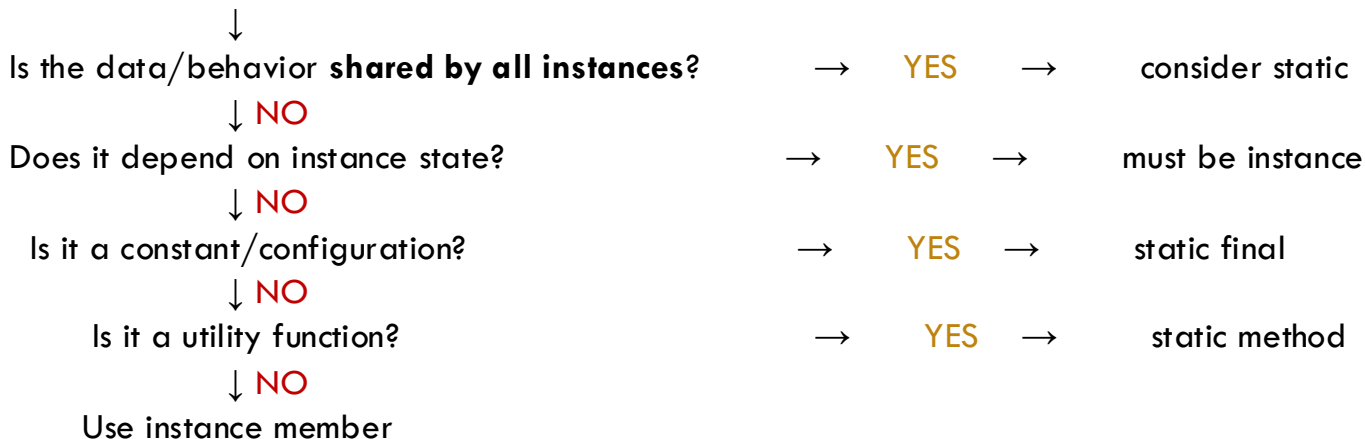


The Employee Class

> Static Fields and Methods

> Decision Flowchart (basic): When to Use Static Members

Start: Should this member be static?



When in doubt: Start with instance members. Only use static when you have a clear, documented reason.



The Employee Class

> **Factory Methods (static object creation)**

>> factory methods are **static methods that create and return objects**

>> used by classes such as `LocalDate` and `NumberFormat`

```
% LocalDate today = LocalDate.now();           // Factory for current date
% LocalDate birthday = LocalDate.of(1990, 5, 15); // Factory with parameters
```

Why use factory methods instead of constructors?

>> **Meaningful names**

>>> constructors must have the class name ... but ...

>>>> factory methods can describe what is created (`getCurrencyInstance`, `getPercentInstance`)

>> **Flexible return types**

>>> Factory methods may return subclasses

>>> Example: returns `DecimalFormat`, a subclass of `NumberFormat`

>> **Instance control** (with a constructor, every call to `new` always creates a new object)

>>> It can reuse or share objects

>>> Example: `Set.of()` may return the same empty set instance (shared instance, immutable, no side effects)



<Exercises>

When should a method be declared static?

- A. When it represents behavior shared by all instances
- B. When it needs to access instance fields
- C. When it uses the 'this' keyword
- D. When it is called only from constructors



<Exercises>

Why is directly exposing instance fields considered bad design?

- A. It slows down the program
- B. It prevents object creation
- C. It requires more memory
- D. It breaks encapsulation and invariants



<Exercises>

What is the main benefit of accessor methods (getters)?

- A. They make code shorter
- B. They allow validation and hide internal representation
- C. They replace constructors
- D. They make fields mutable



<Exercises>

Two objects have identical field values but were created separately. Which statement is correct?

- A. `obj1 == obj2` must be true
- B. `obj1.equals(obj2)` must be false
- C. They have different identities but may be equal in state
- D. They refer to the same memory location



<Exercises>

What does == compare when used on object references in Java?

- A. Object contents
- B. Object identity (memory reference)
- C. Hash codes
- D. Field values



<Exercises>

What is the purpose of the 'this' keyword?

- A. To refer to the current object instance
- B. To create a new object
- C. To access static fields
- D. To infer variable types



<Exercises>

Which of the following is NOT allowed with var?

- A. Local variables
- B. Method parameters
- C. Variables with initializers
- D. Type inference at compile time



Method Parameters

Parameter Passing: Call by Value vs Call by Reference

> Call by value

>> the method receives a **copy of the value**

>> **changes inside the method do not affect the caller's variable**

> Call by reference

>> the **method receives** the location (**reference**) of the variable

>> the method **can modify the caller's variable directly**

> Java's parameter passing model

>> **Java always uses call by value**

>> A method receives a **copy of each argument**

>> A method **cannot modify the contents of variables** used in the method call



Method Parameters

Parameter Passing: Call by Value vs Call by Reference

> Consider the following call:

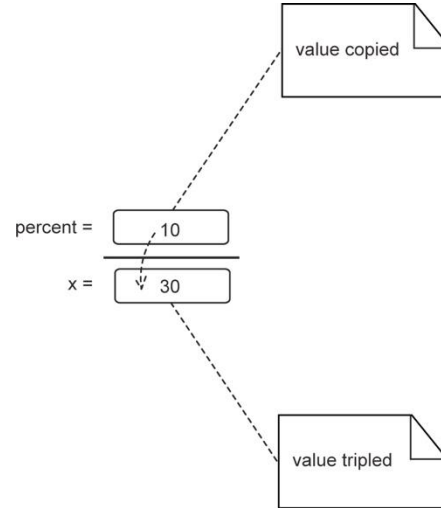
```
% double percent = 10;  
% tripleValue(percent);
```



after the method call, the value of percent remains 10, regardless of any changes made to percent inside the method

```
% void tripleValue(double x) {  
    x = 3 * x;  
}
```

it is impossible for a method to change a primitive type parameter



Method Parameters

Parameter Passing: Call by Value vs Call by Reference

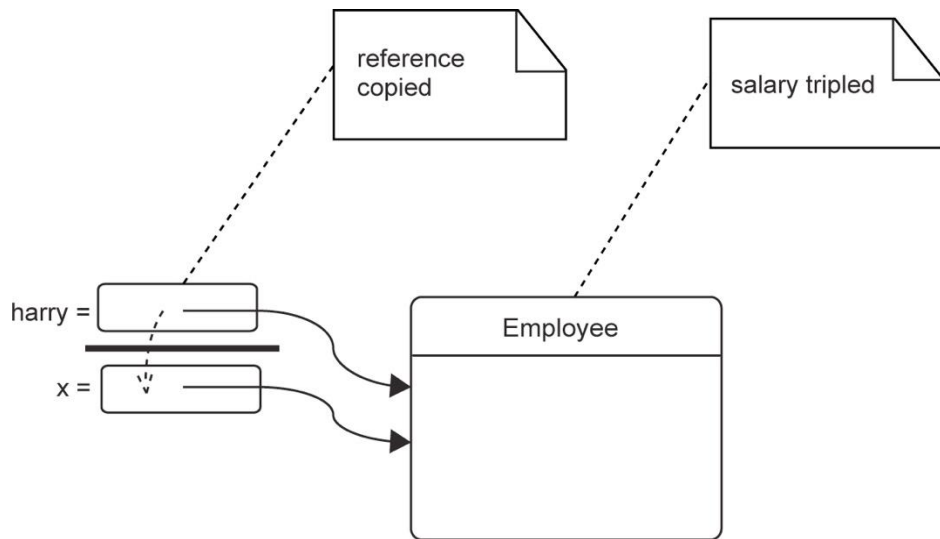
> There are two kinds of parameters: **primitives** and **object references**

>> In the case of objects, the **method gets a copy of the object reference**, and both the original and the copy refer to the same object. So, it is **possible to change the state of an object parameter**

```
% void tripleSalary(Employee x) {  
    x.raiseSalary(200);  
}
```

```
% harry = new Employee(. . .);  
% tripleSalary(harry);
```

Harry's salary is tripled!



In Java, object references are passed by value!



Object Construction

> Overloading

> a class may have more than one constructor (overloading)

>> Java decides which **method to call at compile time** based on the **number, types, or order of parameters**

>>> the process of **finding a match** is called **overloading resolution** (compile-time error if no match is found)

>> **overloading** is supported not only for constructors but also for **any method in Java**

> You can overload a method by changing:

>> number of parameters

>> types of parameters

>> order of parameters

>>> **changing only the return type is not overloading** (not allowed, compile-time error, ambiguity)

> The String class has six public methods called indexOf:

% indexOf(int)	% indexOf(String)
% indexOf(int, int)	% indexOf(String, int)
% indexOf(int, int, int)	% indexOf(String, int, int)

See <https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/lang/String.html>



Object Construction

> Default Field Initialisation

>> if you do **not set a field explicitly in a constructor**, it is **automatically set to a default value**:

>>> numbers to 0,

>>> boolean values to false, and

>>> object references to null

avoid relying on default values; it makes your code harder to understand

> Parameter Names

> Choose names that readers can immediately figure out the meaning of the parameters

>> Never: `int a1; int a2; int abc; int blah;`

> For constructors, you may add “a” as a prefix of the parameter; or use shadow variables (as mentioned)

```
% Employee(String aName, double aSalary) {  
    name = aName;  
    salary = aSalary;  
}
```

```
% Employee(String name, double salary) {  
    this.name = name;  
    this.salary = salary;  
}
```



Records

well, it is shallowly immutable

- > A record is a special form of a class whose state is **immutable** and **readable** by the public
- >> introduced in JDK 14 as a preview feature and fully incorporated into JDK 16
- >> eliminates boilerplate code (constructors, getters, equals, hashCode, toString)
- >> ideal for classes whose main purpose is storing data (data transfer, value objects)

```
% public record Person(String name, int age) {}
```

- > automatically declares instance fields as:

```
% private final String name;  
% private final int age;
```

- > also includes the following canonical constructor:

```
% Person(String name, int age){  
    this.name = name;  
    this.age = age;  
}
```

no need to implement then, they are given to you as part of a record



Records

```
% public record Person(String name, int age) {}
```

```
% Person(String name, int age){  
    this.name = name;  
    this.age = age;  
}
```

> and public accessor methods:

```
% String name();  
% int age();
```

Note that the accessors are called name and age (exactly as the field names)
They do not create getXXX() or isXXX() (for booleans).

```
% var p = new Person("Bernardo", 40);  
% IO.println(p.name() + " is " + p.age());
```

> every record has three methods defined automatically: toString, equals, and hashCode



Records

> records allow the creation of new methods:

```
% public record Person(String name, int age) {  
    public boolean isAdult() {  
        return age >= 18;  
    }  
  
    public String name() {  
        return this.name.toUpperCase();  
    }  
}
```

} new method

} we can also redefine automatically generated methods

> it is **not allowed** to add new instance fields to a record

```
% public record Person(String name, int age) {  
    private final String occupation; //not allowed  
}
```



Records

> **Wait... what is happening here?**

>> Let's take a closer look at what we just did (I hope you asked this question, but if not, here is the answer):

>>> **What is happening with name? Isn't it immutable? Isn't String immutable?**

```
% public record Person(String name, int age) {  
    public String name() {  
        return this.name.toUpperCase();  
    }  
}
```

> the field name in a record is private and final

> redefining name() overrides the automatically generated accessor

> the stored value (change or does not change)?

>> correct answer: does not change

>> why?

>> toUpperCase() creates a new String object

>> only the returned value is transformed, not the internal state

Best practice: keep record accessors unchanged, as overriding them may lead to inconsistencies with other methods that use the instance fields

```
public String upperCaseName() {  
    return name.toUpperCase();  
}
```



Records

> a record, like any class, can have **static fields and methods**:

```
% public record Person(String name, int age) {  
    private static final int MIN_AGE = 18;  
  
    // static factory method  
    public static Person of(String name, int age) {  
        if (!isValidAge(age)) {  
            throw new IllegalArgumentException("Age must be over " + Person.MIN_AGE);  
        }  
        return new Person(name, age); // calls the canonical constructor  
    }  
  
    // static helper method  
    private static boolean isValidAge(int age) {  
        return age >= MIN_AGE;  
    }  
}
```

What happens to the constructor?

Can we still call the constructor?



Records

> we can also **redefine** the canonical constructor

```
% public record Person(String name, int age) {  
    private static final int MIN_AGE = 18;
```

```
    Person {  
        if (!isValidAge(age)) throw new IllegalArgumentException("Age must be over " +  
        Person.MIN_AGE);  
    }  
}
```

used for validation and normalization

What about the instance fields?

```
// static factory method  
public static Person of(String name, int age) {  
    if (!isValidAge(age)) {  
        throw new IllegalArgumentException("Age must be over " + Person.MIN_AGE);  
    }  
    return new Person(name, age);  
}  
}
```

Should we remove this?



Records

In summary

- > all instance fields are **final**
- > records are **immutable** (more precisely, **shallowly immutable**)
- > **no setters** are allowed
- > the object's **state cannot change after construction**
- > use records for **data-centric classes** (data matters more than behavior)
- > the following are **automatically provided**:
 - >> accessor methods
 - >> canonical constructor
 - >> equals(), hashCode(), toString()
- > records **significantly reduce boilerplate code**

- > records **do not** extend user-defined classes (they implicitly extend java.lang.Record)
- > records **cannot** be extended (they are implicitly final)
- > records **can** implement interfaces



Access Modifiers (Classes & Methods)

> Control visibility of classes, methods, and variables

Modifier	Class	Package	Subclass	World
public	✓	✓	✓	✓
protected	✓	✓	✓	✗
default (no modifier specified)	✓	✓	✗	✗
private	✓	✗	✗	✗

For Classes

accessible everywhere

not allowed (inner classes only)

within the same package

not allowed (inner classes only)



Access Modifiers (Classes & Methods)

> For methods and instance fields

```
% public class Employee {  
    public String name;           // Everywhere  
    protected int id;            // Package + subclasses  
    String department;           // Package only (default)  
    private double salary;        // Class only  
  
    public void calculatePay() { } // Everywhere  
    protected void promote() { }  // Package + subclasses  
    void internalProcess() { }     // Package only  
    private void validateSalary() { } // Class only  
}
```

As a general guideline, use the most restrictive access level possible to preserve encapsulation

- > Use private for internal implementation details, instance fields (with public accessors/mutators) and helper methods
- > Use public for API contracts, interface methods, main entry points
- > Do not worry with package (default) and protected for now:
 - >> Use default for package-internal utilities and classes/methods only used within package
 - >> Use protected for methods for subclasses to override, library extension points



Class Design Hints

> Always keep data private

- >> it preserves encapsulation
- >> it allows internal representation to change safely (without breaking code)
- >> it makes bugs easier to detect

> Always initialize data

- >> do not rely on default values (initialize fields explicitly)
- >> ensure objects are always in a valid state

> Don't use too many basic types in a class

```
% String street;  
% String city;  
% String state;  
% int zip;  
% Address address;
```

—————→

>> easier to evolve and maintain (more readable)



Class Design Hints

> Not all fields need individual field accessors and mutators

>> some fields should be read-only

>> some fields should be internal only

>>> Example: hiringDate should be read-only

> Break up classes that have too many responsibilities

>> Each class should model one concept

>> Classes with too many responsibilities are hard to maintain

>> Single Responsibility Principle

> Make the names of your classes and methods reflect their responsibilities

>> Class names: nouns (Order), adjective + nouns (RushOrder), gerunds (BillingAddress)

>> Method names: Accessors (getX), Mutators (setX)

> Prefer immutable classes

>> safer in concurrent environments

>> easy to reason about





```
public class Student {  
    public String name;  
    public int marks;  
  
    public Student(String name, int marks) {  
        name = name;  
        marks = marks;  
    }  
}
```



<Exercises>

What is true about static fields?

- A. Each object has its own copy
- B. They are initialized when an object is created
- C. There is only one copy per class
- D. They cannot be final



<Exercises>

What is a key characteristic of a record in Java?

- A. It is mutable
- B. It allows adding new instance fields
- C. It automatically provides equals(), hashCode(), and toString()
- D. It can extend any user-defined class



<Exercises>

Why might a factory method be preferred over a constructor?

- A. Constructors cannot take parameters
- B. Factory methods can control object creation and reuse instances
- C. Constructors always return null
- D. Factory methods must return new objects



<Exercises>

Which statement best describes parameter passing in Java?

- A. Java uses call by reference
- B. Java uses call by value for primitives only
- C. Objects are passed by reference
- D. Java always uses call by value



<Exercises>

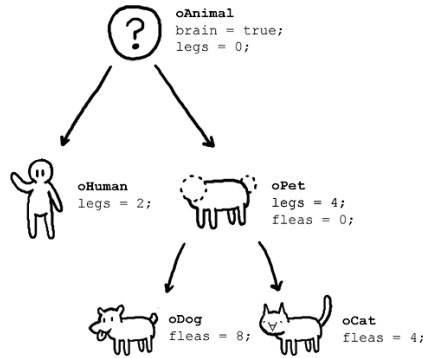
Which access modifier is most restrictive?

- A. public
- B. protected
- C. Default (package-private)
- D. private

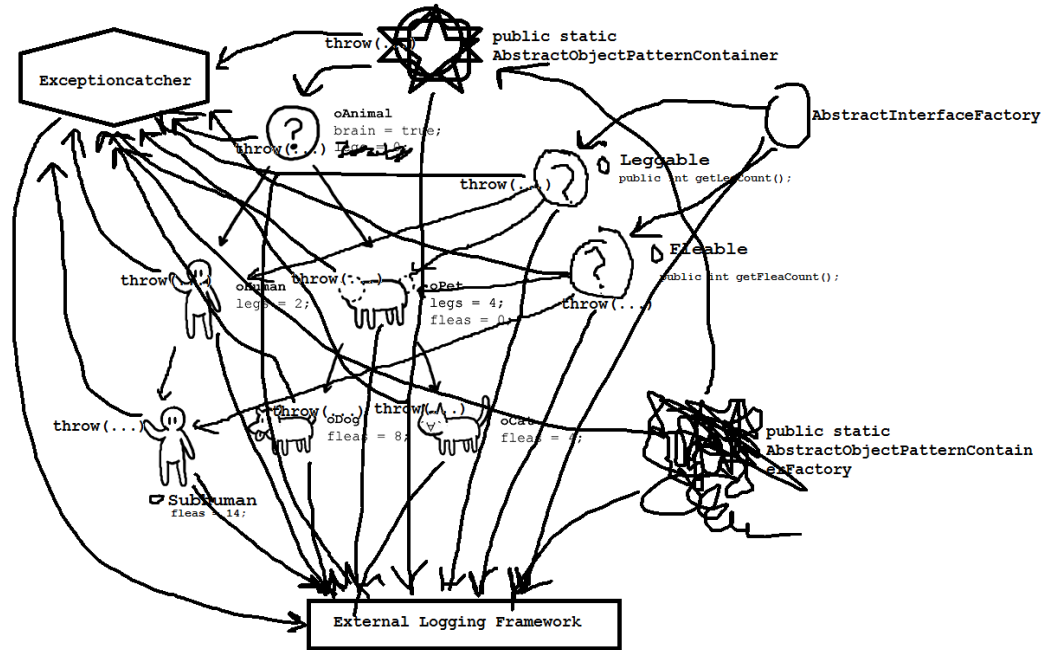


MEME

What OOP users claim



What actually happens



Thank you



Australian
National
University

TEQSA PROVIDER ID: PRV12002 (AUSTRALIAN UNIVERSITY)

CRICOS PROVIDER CODE: 00120C